# Contest (1)

## template.cpp

**Description:** Base file for every solution. `int` is `long long`, so use `int32_t main`. Globals in this notebook must be (re)assigned inside `solve()` for each test.
**How it works:** `#define int long long` turns every `int` into 64 bits, so sums and products up to about $9 \cdot 10^{18}$ are safe; that is why `main` must be written `int32_t main`. `solve()` runs once per test case.
**Modify / tips:** problem with a single test: delete the line `cin >> tt;`. Memory limit hit by a huge array: declare that array as `vector<int32_t>`. Answer modulo something: take `% MOD` after *every* multiplication and addition, and write `((x % MOD) + MOD) % MOD` after a subtraction.

20 lines

```
#pragma GCC optimize("O3")
#pragma GCC optimize("unroll-loops")
#include <bits/stdc++.h>
#define int long long
using namespace std;
#define MOD 1000000007
#define MOD2 998244353

void solve() {
  // REMEMBER TO ASSIGN IF NEEDED!!!!!!
}
int32_t main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);
  int tt = 1;
  cin >> tt;
  while (tt--) {
    solve();
  }
}
```

## BitTricks.h

**Description:** 64-bit builtins and single-bit operations. Use for bitmask DP, subset iteration, finding the lowest/highest set bit. `clz`/`ctz` are undefined for $x = 0$.
**How it works:** a set of up to 62 items is stored in one number: bit $i$ is 1 when item $i$ is in the set. All subsets of $n$ items are the numbers $0..2^n - 1$: `for (int mask = 0; mask < (1LL << n); mask++)`.
**Modify / tips:** always write `1LL << k`, never `1 << k` (overflows for $k \geq 31$). XOR facts: $x \oplus x = 0$, a prefix-XOR array gives the XOR of a range like prefix sums.

22 lines

```
// number of 1-bits in x
__builtin_popcountll(x);
// 1 if popcount is odd, 0 if even
__builtin_parityll(x);
// count leading zeros in 64-bit
__builtin_clzll(x);   // x != 0
// count trailing zeros (use: lowest set bit, bit iteration)
__builtin_ctzll(x);   // x != 0
// index of highest 1-bit (use: power of 2 <= x)
int msb = 63 - __builtin_clzll(x);
// index of lowest 1-bit
int lsb = __builtin_ctzll(x);
// check if power of two
(x > 0 && (x & (x - 1)) == 0);
// check k-th bit
(x >> k) & 1;
// set k-th bit
x | (1LL << k);
// toggle k-th bit
x ^ (1LL << k);
// clear k-th bit
x & ~(1LL << k);
```

## CustomSort.h

**Description:** Sorting by any rule. A comparator is a function that takes two elements and answers one question: "must `a` come before `b`?". Pass its name as the third argument of `sort`. The same pattern covers every case: compare the most important key first; only if it is equal go to the next key.
**How it works:** `sort` calls the comparator on pairs of elements and places `a` first whenever it returns true. **Rule that must never be broken:** for two equal elements it has to return *false* (use `<` or `>`, never `<=` or `>=`), otherwise `sort` may crash or loop forever.
**Modify / tips:** flip `<` to `>` on a key to make that key descending. Built-in shortcuts: `sort(v.rbegin(), v.rend())` = descending; pairs and vectors already sort by first element, then second, and so on, so storing `{key, value}` pairs (or `-key` for descending) often avoids a comparator. `stable_sort` keeps the input order of equal elements. Sort by a fraction $a.x/a.y$ without doubles: `return a.x * b.y < b.x * a.y;` (positive $y$). Arrange numbers as strings to form the largest number: `return a + b > b + a;`.
**Usage:** `sort(v.begin(), v.end(), by_second);`
**Time:** $O(n \log n)$ comparisons

51 lines

```
// (1) pairs: by second value, ties by first value
bool by_second(const pair<int,int>& a,
    const pair<int,int>& b) {
  if (a.second != b.second) return a.second < b.second;
  return a.first < b.first;
}
// (2) list of groups (cycles, components, paths): the
// biggest group first, equal sizes: smaller content first
bool by_size(const vector<int>& a, const vector<int>& b) {
  if (a.size() != b.size()) return a.size() > b.size();
  return a < b;
}
// (3) records with several keys
struct Student {
  string name;
  int marks, age;
};
bool by_marks(const Student& a, const Student& b) {
  if (a.marks != b.marks) return a.marks > b.marks; // high
  if (a.age != b.age) return a.age < b.age; // then young
  return a.name < b.name; // then alphabetical
}
// (4) sort positions, not values: order[k] = index of the
// k-th smallest element (original array stays untouched)
vector<int> sorted_indices(vector<int>& a) {
  int n = a.size();
  vector<int> order(n);
  for (int i = 0; i < n; i++) order[i] = i;
  // a comparator written in place; [&] lets it read a
  sort(order.begin(), order.end(), [&](int i, int j) {
    if (a[i] != a[j]) return a[i] < a[j]; // MODIFY: the key
    return i < j;
  });
  return order;
}
// (5) custom order inside set / map / priority_queue needs
// a struct with operator() instead of a function
struct BySecond {
  bool operator()(const pair<int,int>& a,
      const pair<int,int>& b) const {
    if (a.second != b.second) return a.second < b.second;
    return a.first < b.first;
  }
};
// set<pair<int,int>, BySecond> s;
//   *s.begin() = smallest second. Elements for which
//   neither comes before the other count as duplicates
// priority_queue<pair<int,int>, vector<pair<int,int>>,
//   BySecond> pq;
//   pq.top() = the LAST element of this order (largest
//   second). For smallest on top flip < to > in BySecond
```

## CustomHash.h

**Description:** Hash for `unordered_map` / `unordered_set`. Needed in two situations: (a) the key is a `pair` – C++ has no built-in hash for pairs, so it does not even compile without one; (b) keys are integers and the tests are built to break the default hash (common on Codeforces), which turns every operation into $O(n)$ and gives TLE. When neither speed nor (a) matters, a plain `map` / `set` is always safe and needs nothing: it works with pairs, vectors and strings as keys in $O(\log n)$.
**How it works:** the key is added to a number taken from the clock when the program starts (nobody can predict it) and then scrambled by `splitmix64`, so no prepared input can make many keys collide.
**Modify / tips:** for a key with three numbers scramble them one after another like the pair version. For vector / string keys use `map`.
**Usage:** `unordered_map<int, int, custom_hash> mp;`
**Time:** $O(1)$ average per operation

23 lines

```
const uint64_t HASH_SEED =
  chrono::steady_clock::now().time_since_epoch().count();
struct custom_hash {
  static uint64_t splitmix64(uint64_t x) { // scrambles bits
    x += 0x9e3779b97f4a7c15;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
    x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
    return x ^ (x >> 31);
  }
  size_t operator()(uint64_t x) const { // integer keys
    return splitmix64(x + HASH_SEED);
  }
  size_t operator()(const pair<int,int>& p) const { // pairs
    uint64_t h = splitmix64(p.first + HASH_SEED);
    return splitmix64(h + p.second);
  }
};
// unordered_map<int, int, custom_hash> mp;
// unordered_map<pair<int,int>, int, custom_hash> cell;
// unordered_set<pair<int,int>, custom_hash> seen;
// mp.reserve(1 << 20);  optional: avoids slow regrowing
// if (mp.count(x)) ...  check a key WITHOUT creating it
//                       (mp[x] creates the key with 0)
```

# Mathematics (2)

## ModPow.h

**Description:** Binary exponentiation, $base^{exp}$ mod $M$. Use for modular powers and for inverses under a prime modulus: $a^{-1} = a^{M-2}$. Needs $base < M$ and $M^2$ to fit in 64 bits.
**How it works:** write `exp` in binary. `base` becomes $base^1, base^2, base^4, base^8, \ldots$ by squaring; multiply it into the result whenever the current bit of `exp` is 1.
**Modify / tips:** division under a prime modulus: $x/a \equiv x \cdot$ `binExp(a, M - 2, M)`. For $M$ near $10^{18}$ use `pow_mod` from MillerRabin. Huge exponent $b^c$: reduce it as `binExp(a, binExp(b, c, M - 1), M)` ($M$ prime).
**Time:** $O(\log exp)$

9 lines

```
int binExp(int base, int exp, int M) { // base^exp % M
  int result = 1;
  while (exp > 0) {
    if (exp % 2 == 1) result = (result * base) % M;
    base = (base * base) % M; // next power of two
    exp /= 2;
  }
  return result;
}
```

## ModInverse.h

**Description:** Modular inverse by extended Euclid. Works for any modulus (not only primes); returns $-1$ when $\gcd(a, M) \neq 1$. Use when $M$ is composite or unknown.
**How it works:** runs Euclid's algorithm on $(a, M)$ while tracking a number $u$ with $a \cdot u \equiv$ (current remainder) mod $M$; when the remainder reaches 1, $u$ is the inverse.
**Modify / tips:** to divide under a modulus multiply by the inverse: `x * modinv(a, M) % M`. An inverse exists only if $\gcd(a, M) = 1$.
**Time:** $O(\log M)$

12 lines

```
int modinv(int a, int M) { // a^-1 mod M, -1 if none
  int b = M, u = 1, v = 0;
  while (b) {
    int t = a / b;
    a -= t * b; swap(a, b);
    u -= t * v; swap(u, v);
  }
  if (a != 1) return -1; // gcd != 1: no inverse
  u %= M;
  if (u < 0) u += M;
  return u;
}
```

## Factorial.h

**Description:** Factorials and inverse factorials mod prime $M$, giving $\binom{n}{r}$ and ${}^nP_r$ in $O(1)$. Use for any counting problem; needs $N < M$. Requires `binExp`.
**How it works:** $\binom{n}{r} = \frac{n!}{r!\,(n-r)!}$. Division mod $M$ is multiplication by an inverse, so store inverse factorials: one `binExp` gives `invfact[N]`, then `invfact[i] = invfact[i+1] * (i+1)`.
**Modify / tips:** call `build_fact` once in `main` with the largest $n$ ($2 \cdot 10^5$ or $10^6$), not per test. Formulas: split $n$ equal items into $k$ boxes $= \binom{n+k-1}{k-1}$; grid paths right/down $= \binom{r+c}{r}$; Catalan $= \binom{2n}{n}/(n+1)$; arrangements with repeats $= n!/(c_1!\,c_2!\cdots)$.
**Usage:** `build_fact(N, MOD); nCr(n, r, MOD);`
**Time:** $O(N + \log M)$ build, $O(1)$ per query

21 lines

```
vector<int> fact, invfact;
void build_fact(int N, int M) { // fact[], invfact[] up to N
  fact = vector<int>(N + 1, 0);
  invfact = vector<int>(N + 1, 0);
  fact[0] = 1;
  for (int i = 1; i <= N; i++) {
    fact[i] = fact[i - 1] * i % M;
  }
  invfact[N] = binExp(fact[N], M - 2, M); // 1 / N!
  for (int i = N - 1; i >= 0; i--) {
    invfact[i] = invfact[i + 1] * (i + 1) % M;
  }
}
int nCr(int n, int r, int M) { // choose r of n, order ignored
  if (r < 0 || r > n) return 0;
  return fact[n] * invfact[r] % M * invfact[n - r] % M;
}
int nPr(int n, int r, int M) { // arrange r of n, order matters
  if (r < 0 || r > n) return 0;
  return fact[n] * invfact[n - r] % M;
}
```

## Matrix.h

**Description:** Matrix product and fast power mod `MOD`. Use for linear recurrences (Fibonacci-like, $k$-term DP transitions) and counting walks of length $exp$ in a graph.
**How it works:** if every new value is a fixed linear mix of the previous few values, write one step as (new vector) $= T \cdot$ (old vector). Then $k$ steps are $T^k$, computed by squaring like `binExp`. Fibonacci: $T = \{\{1,1\},\{1,0\}\}$ and $F_k = T^k[0][1]$.
**Modify / tips:** row $i$ of $T$ says how new element $i$ is built from the old elements. For $f_n = af_{n-1} + bf_{n-2} + c$ use the vector $(f_{n-1}, f_{n-2}, 1)$ and $T = \{\{a,b,c\},\{1,0,0\},\{0,0,1\}\}$. Number of walks of length $k$ from $u$ to $v$ = (adjacency matrix)$^k[u][v]$. Keep the size $\leq 100$.
**Usage:** `auto R = matrixExp(T, k);`
**Time:** $O(n^3)$ multiply, $O(n^3 \log exp)$ power

30 lines

```
vector<vector<int>> identityMatrix(int n) { // n x n identity
  vector<vector<int>> I(n, vector<int>(n, 0));
  for (int i = 0; i < n; i++) I[i][i] = 1;
  return I;
}
vector<vector<int>> matrixMultiply(vector<vector<int>> &A,
    vector<vector<int>> &B) { // A * B mod MOD
  int n = A.size(), m = B[0].size(), K = A[0].size();
  vector<vector<int>> product(n, vector<int>(m, 0));
  for (int i = 0; i < n; i++) {
    for (int k = 0; k < K; k++) {
      if (A[i][k] == 0) continue;
      for (int j = 0; j < m; j++) {
        product[i][j] =
          (product[i][j] + A[i][k] * B[k][j]) % MOD;
      }
    }
  }
  return product;
}
vector<vector<int>> matrixExp(vector<vector<int>> base,
    int exp) { // base^exp mod MOD
  vector<vector<int>> result = identityMatrix(base.size());
  while (exp > 0) {
    if (exp & 1) result = matrixMultiply(result, base);
    base = matrixMultiply(base, base);
    exp >>= 1;
  }
  return result;
}
```

## Gcd.h

**Description:** Euclid's gcd and overflow-safe lcm (divides before multiplying). Use for fraction reduction, periods, divisibility arguments.
**How it works:** $\gcd(a, b) = \gcd(b, a \bmod b)$ until $b = 0$.
**Modify / tips:** `lcm` can overflow when the answer exceeds $9 \cdot 10^{18}$; cap it if the problem only needs "$>$ limit". gcd of a whole array: fold `g = gcd(g, a[i])` starting from 0.
**Time:** $O(\log \min(a, b))$

6 lines

```
int gcd(int a, int b) { // greatest common divisor
  return (!b ? a : gcd(b, a % b));
}
int lcm(int a, int b) { // least common multiple
  return a / gcd(a, b) * b;
}
```

## IntRoot.h

**Description:** $\lfloor \sqrt[y]{x} \rfloor$ by binary search with `__int128`, no floating point error. Use for exact square/cube roots and perfect-power checks up to $10^{18}$.
**How it works:** binary search for the largest $m$ with $m^y \leq x$. The power is built one multiplication at a time in `__int128` and stopped as soon as it exceeds $x$, so nothing overflows.
**Modify / tips:** this is the general *binary search on the answer* skeleton: keep `l, h, ans`, replace the block that sets `ok` by any check that is true for small `m` and false for large `m` ("can we finish with capacity $m$", "can we place cows at distance $m$"). If the check is false-then-true, swap the two branches.
**Time:** $O(y \log x)$

27 lines

```
int getRoot(int x, int y) { //y-th root of x
  int l = 0, h = x, ans = 0;
  while (l <= h) {
    int m = l + (h - l) / 2;
    __int128 cur = 1;
    bool ok = true;
    for (int i = 0; i < y; i ++) {
      cur *= m;
      if (cur > x) {
        ok = false;
        break;
      }
    }
    if (ok) {
      ans = m;
      l = m + 1;
    } else {
      h = m - 1;
    }
  }
  return ans;
}
bool isPerfectSquare(int n) { // is n = r * r
  if (n < 0) return false;
  int r = getRoot(n, 2);
```

```
  return r * r == n;
}
```

## PrefixSums.h

**Description:** `pref[i]` = sum of the first $i$ elements, `suff[i]` = sum of the last $i$ elements. Use for $O(1)$ static range sums: sum of $a[l..r]$ (0-indexed) is `pref[r+1] - pref[l]`.
**How it works:** `pref[i]` is the sum of everything before position $i$, so a range sum is a difference of two prefix values.
**Modify / tips:** 2D: `p[i][j] = a[i-1][j-1] + p[i-1][j] + p[i][j-1] - p[i-1][j-1]` and the sum of rows $r_1..r_2$, columns $c_1..c_2$ (1-indexed) is `p[r2][c2] - p[r1-1][c2] - p[r2][c1-1] + p[r1-1][c1-1]`. Many range additions, then read once: `diff[l] += x; diff[r+1] -= x;` and take the prefix sum of `diff`. Count subarrays with sum $k$: map of prefix sums seen so far.
**Time:** $O(n)$

18 lines

```
// pref[i] = sum of first i
vector<int> prefixArr(vector<int>& arr) {
  int n = arr.size();
  vector<int> pref(n + 1, 0);
  for (int i = 1; i <= n; i ++) {
    pref[i] = pref[i - 1] + arr[i - 1];
  }
  return pref;
}
// suff[i] = sum of last i
vector<int> suffixArr(const vector<int>& arr) {
  int n = arr.size();
  vector<int> suff(n + 1, 0);
  for (int i = 1; i <= n; i ++) {
    suff[i] = suff[i - 1] + arr[n - i];
  }
  return suff;
}
```

# Number theory (3)

## Sieve.h

**Description:** Sieve of Eratosthenes; `prime[i]` is true iff $i$ is prime. Use when you only need primality of all numbers up to $n$ ($n \le 10^7$ or so).
**How it works:** every composite number has a prime factor $\le \sqrt{n}$, so for each prime $p$ cross out its multiples starting from $p^2$.
**Modify / tips:** to get the list of primes, loop over the result and push every $i$ with `prime[i]`. Build it once in `main`, not per test.
**Time:** $O(n \log \log n)$

11 lines

```
vector<bool> sieve(int n) { // prime[i] = is i prime
  vector<bool> prime(n + 1, true);
  prime[0] = prime[1] = false;
  for (int p = 2; p * p <= n; ++p) {
    if (prime[p] == true) {
      for (int i = p * p; i <= n; i += p) // multiples of p
        prime[i] = false;
    }
  }
  return prime;
}
```

## SPF.h

**Description:** Smallest-prime-factor sieve. `primeFactors(n)` returns the *distinct* primes of $n \le N$. Use to factorise many numbers quickly ($spf[x] = x$ iff $x$ is prime).
**How it works:** `spf[x]` = smallest prime dividing $x$. Dividing $x$ by `spf[x]` again and again peels off its prime factors one at a time.
**Modify / tips:** to get exponents, count the iterations of the inner `while`; number of divisors $= \prod(e_i + 1)$, sum of divisors $= \prod(p^{e_i+1} - 1)/(p - 1)$. Build once in `main` with $N$ = largest value in the input.
**Usage:** `buildSPF(N); auto pf = primeFactors(x);`
**Time:** $O(N \log \log N)$ build, $O(\log n)$ per query

27 lines

```
vector<int> spf;
void buildSPF(int N) { // spf[i] = smallest prime of i
  spf.resize(N + 1);
  for (int i = 0; i <= N; i++) {
    spf[i] = i;
  }
  for (int i = 2; i * i <= N; i++) {
    if (spf[i] == i) {
      for (int j = i * i; j <= N; j += i) {
        if (spf[j] == j) {
          spf[j] = i;
        }
      }
    }
  }
}
vector<int> primeFactors(int n) { // distinct primes of n
  vector<int> pf;
  while (n > 1) {
    int p = spf[n];
    pf.push_back(p);
    while (n % p == 0) {
      n /= p;
    }
  }
  return pf;
}
```

## Phi.h

**Description:** Euler's totient $\varphi(n)$ = count of $1 \le k \le n$ coprime to $n$. `getPhi` for one large $n$, `buildPhi` for all $n \le N$. Use for $a^{\varphi(m)} \equiv 1$, exponent reduction, counting coprime pairs / gcd sums.
**How it works:** $\varphi(n) = n \prod(1 - 1/p)$ over the distinct primes $p$ of $n$; each `result -= result / p` applies one factor.
**Modify / tips:** $\sum_{d|n} \varphi(d) = n$; the count of $k \le n$ with $\gcd(k, n) = g$ is $\varphi(n/g)$. Power towers: reduce the exponent mod $\varphi(M)$.
**Time:** $O(\sqrt{n})$ single, $O(N \log \log N)$ sieve

27 lines

```
int getPhi(int n) { // phi of one n
  int result = n;
  for (int i = 2; i * i <= n; i++) {
    if (n % i == 0) {
      while (n % i == 0)
        n /= i;
      result -= result / i; // multiply by (1 - 1/i)
    }
  }
  if (n > 1) result -= result / n;
  return result;
}
vector<int> phi_arr;
void buildPhi(int N) { // phi_arr[i] for all i <= N
  phi_arr.resize(N + 1);
  for (int i = 0; i <= N; i++) {
    phi_arr[i] = i;
  }
  for (int i = 2; i <= N; i++) {
    if (phi_arr[i] == i) {
      // i is a prime number
      for (int j = i; j <= N; j += i) {
        phi_arr[j] -= phi_arr[j] / i;
      }
    }
  }
}
```

## Divisors.h

**Description:** All divisors of $n$ by trial division, *unsorted*. Use to enumerate divisors of a single number up to about $10^{12}$.
**How it works:** divisors come in pairs $(i, n/i)$ with $i \le \sqrt{n}$.
**Modify / tips:** call `sort` on the result if order matters. For all numbers up to $N$ at once use a harmonic loop: `for i in 1..N, for j = i, 2i, .. : divs[j].push_back(i)` in $O(N \log N)$.
**Time:** $O(\sqrt{n})$

11 lines

```
vector<int> factors(int n) { // all divisors, unsorted
  vector<int> f;
  for (int i = 1; i * i <= n; i++) {
    if (n % i == 0) {
      f.push_back(i);
      if (i != n / i)
        f.push_back(n / i);
    }
  }
  return f;
}
```

## MillerRabin.h

**Description:** Exact primality test for one number up to $10^{18}$, far beyond the sieve ($10^7$) and trial division ($10^{12}$, slow). Use when the input has huge numbers and asks "is it prime", "next prime after $n$" (test $n+1, n+2, \ldots$; gaps are small), or when there are many queries on numbers around $10^9..10^{12}$.
**How it works:** write $n-1 = odd \cdot 2^r$. For a prime $n$ and any base $a$, the sequence $a^{odd}, a^{2 \cdot odd}, \ldots$ must start with 1 or hit $n-1$ before reaching 1. A composite $n$ fails this for some base; the 12 fixed bases below are proven enough for every 64-bit number, so the answer is always correct. Products are done in `__int128` because $n^2$ overflows 64 bits.
**Time:** $O(12 \log n)$ per number

40 lines

```
int mul_mod(int a, int b, int M) { // a * b % M, no overflow
  return (__int128)a * b % M;
}
int pow_mod(int base, int exp, int M) { // base^exp % M
  int result = 1;
  base %= M;
  while (exp > 0) {
    if (exp % 2 == 1) result = mul_mod(result, base, M);
    base = mul_mod(base, base, M);
    exp /= 2;
  }
  return result;
```

```
}
bool is_prime(int n) {
  if (n < 2) return false;
  vector<int> bases = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
    31, 37};
  for (int p : bases) { // small primes settle small n
    if (n % p == 0) return n == p;
  }
  int odd = n - 1, r = 0;
  while (odd % 2 == 0) { // n - 1 = odd * 2^r
    odd /= 2;
    r++;
  }
  for (int a : bases) {
    int x = pow_mod(a, odd, n);
    if (x == 1 || x == n - 1) continue; // passes this base
    bool composite = true;
    for (int i = 1; i < r; i++) {
      x = mul_mod(x, x, n);
      if (x == n - 1) {
        composite = false;
        break;
      }
    }
    if (composite) return false; // a proves n is not prime
  }
  return true;
}
```

## ExtGcdCRT.h

**Description:** `ext_gcd` finds integers $x, y$ with $ax + by = \gcd(a, b)$. `solve_equation` solves $ax + by = c$ (possible only when $\gcd(a, b)$ divides $c$): use for "reach exactly $c$ using steps / jugs / coins of sizes $a$ and $b$, negatives allowed", and for counting or minimising solutions. `crt` merges two conditions $x \equiv r_1 \pmod{m_1}$, $x \equiv r_2 \pmod{m_2}$ into one $x \equiv r \pmod{lcm}$: use for "two periodic events, when do they happen together", "number that leaves given remainders". The moduli do not have to be coprime. Inputs must be non-negative, $a, b$ not both 0.
**How it works:** Euclid's step $\gcd(a, b) = \gcd(b, a \bmod b)$ run backwards: from a solution for $(b, a \bmod b)$ build the one for $(a, b)$. From one solution $(x, y)$ of the equation all others are $x + k \cdot b/g$, $y - k \cdot a/g$ for any integer $k$ (pick $k$ to make $x$ smallest non-negative: `x = ((x % (b/g)) + b/g) % (b/g)`). CRT writes $x = r_1 + m_1 k$ and solves $m_1 k \equiv r_2 - r_1 \pmod{m_2}$ with `ext_gcd`.
**Modify / tips:** for more than two congruences fold them: `cur = crt(cur.first, cur.second, r[i], m[i])`, stop if it returns $-1$.
**Time:** $O(\log \min(a, b))$

37 lines

```
// returns g = gcd(a, b), sets x, y so that a*x + b*y = g
int ext_gcd(int a, int b, int& x, int& y) {
  if (b == 0) {
    x = 1; // a*1 + 0*0 = a
    y = 0;
    return a;
  }
  int x1, y1;
  int g = ext_gcd(b, a % b, x1, y1); // b*x1 + (a%b)*y1 = g
  x = y1;
  y = x1 - (a / b) * y1;
  return g;
}
// one solution of a*x + b*y = c, false if there is none
bool solve_equation(int a, int b, int c, int& x, int& y) {
  int g = ext_gcd(a, b, x, y);
  if (c % g != 0) return false;
  x *= c / g;
  y *= c / g;
  return true;
}
// x = r1 (mod m1) and x = r2 (mod m2), 0 <= r1 < m1 etc.
// returns {x, lcm(m1, m2)} with 0 <= x < lcm; every other
// answer is x + k * lcm. Returns {-1, -1} if impossible.
pair<int,int> crt(int r1, int m1, int r2, int m2) {
  int p, q;
  int g = ext_gcd(m1, m2, p, q);
  if ((r2 - r1) % g != 0) return {-1, -1};
  int l = m1 / g * m2; // lcm, must fit in long long
  int step = m2 / g;
  // x = r1 + m1 * k where k = (r2 - r1) / g * p mod step
  int k = (__int128)((r2 - r1) / g % step) * (p % step)
    % step;
  int x = ((__int128)m1 * k + r1) % l;
  if (x < 0) x += l;
  return {x, l};
}
```

# Data structures (4)

## OrderedSet.h

**Description:** GNU policy tree (PBDS) = a `set` that also answers "how many elements are smaller than $x$" and "what is the $k$-th smallest" in $O(\log n)$. Use for inversion counts, rank / $k$-th queries with insertions and deletions, sliding-window medians. Works on CodeChef, Codeforces, AtCoder (any GCC). With this `using`, do not name a variable `tree`.
**Time:** $O(\log n)$ per operation

27 lines

```
// put these 3 lines right after #include <bits/stdc++.h>
// (BEFORE #define int long long)
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
// put the typedefs after #define int long long
typedef tree<int, null_type, less<int>, rb_tree_tag,
  tree_order_statistics_node_update> ordered_set;
// ordered_set s;         (no duplicates, like set<int>)
// s.insert(x);  s.erase(x);  s.size();  s.find(x);
// s.order_of_key(x);     number of elements < x
// *s.find_by_order(k);   k-th smallest, k starts from 0
//                        (only valid when k < s.size())
// s.lower_bound(x), s.upper_bound(x) work like in set

// duplicates: store {value, unique id}
typedef tree<pair<int,int>, null_type, less<pair<int,int>>,
  rb_tree_tag, tree_order_statistics_node_update>
  ordered_multiset;
// ordered_multiset ms;  int id = 0;
// ms.insert({x, id++});
// ms.order_of_key({x, 0});           elements < x
// ms.order_of_key({x, LLONG_MAX});   elements <= x
// ms.find_by_order(k)->first;        k-th smallest value
// erase ONE copy of x:
// auto it = ms.lower_bound({x, 0});
// if (it != ms.end() && it->first == x) ms.erase(it);
```

## DSU.h

**Description:** Disjoint set union with path compression and union by size. Use for connectivity under edge additions, Kruskal's MST, counting components, offline merging.
**How it works:** each set is a tree; `parent[v]` points one step towards the set's leader (the root). `find_set` walks to the root and re-points everything on the way directly to it. `union_sets` hangs the smaller tree under the larger.
**Modify / tips:** to keep data per set (sum, min, edge count) store it at the leader and combine it inside `union_sets` next to `sz[a] += sz[b]`. Number of components: start at $n$, subtract 1 on every successful union. Cycle in an undirected graph: an edge whose two ends already have the same leader. Kruskal MST: sort edges by weight, take an edge if its ends have different leaders. Edges are only ever added; for deletions process the queries backwards.
**Usage:** `parent.resize(n+1); sz.resize(n+1); make_set(v) for all v;`
**Time:** $O(\alpha(n))$ amortised per operation

19 lines

```
vector<int> parent;
vector<int> sz;
void make_set(int v) { // v alone in a new set
  parent[v] = v;
  sz[v] = 1;
}
int find_set(int v) { // leader of the set of v
  if (v == parent[v]) return v;
  return parent[v] = find_set(parent[v]); // point to leader
}
void union_sets(int a, int b) { // merge sets of a and b
  a = find_set(a);
  b = find_set(b);
  if (a != b) {
    if (sz[a] < sz[b]) swap(a, b); // a = bigger set
    parent[b] = a;
    sz[a] += sz[b];
  }
}
```

## Fenwick.h

**Description:** Fenwick tree (BIT), *1-indexed*: point add, prefix sum. Sum of $[l, r]$ is `query(r) - query(l-1)`. Use for dynamic prefix sums, inversion counting, order statistics on compressed values.
**How it works:** `bit_arr[i]` holds the sum of the last `(i & -i)` elements ending at $i$. A prefix is covered by $O(\log n)$ such blocks, and one element lies in $O(\log n)$ blocks.
**Modify / tips:** range add + single element read: `fenwick_update(l, x); fenwick_update(r + 1, -x);` then `fenwick_query(i)` is $a[i]$. Inversion count: compress values to $1..n$, go right to left, add `fenwick_query(a[i] - 1)`, then `fenwick_update(a[i], 1)`. A 0-indexed array must be shifted by 1.
**Usage:** `bit_n = n; bit_arr.assign(bit_n + 1, 0);`
**Time:** $O(\log n)$ per operation

16 lines

```
// bit_arr[i] = sum of a block of a[] ending at i
// sum of a[l..r] = fenwick_query(r) - fenwick_query(l - 1)
int bit_n;
vector<int> bit_arr;
// bit_n = n;
// bit_arr.assign(bit_n + 1, 0);
void fenwick_update(int i, int val) { // a[i] += val, 1-indexed
```

```
  // i & -i = lowest set bit: go to next block holding i
  for (; i <= bit_n; i += i & -i) bit_arr[i] += val;
}
int fenwick_query(int i) { // a[1] + ... + a[i]
  int sum = 0;
  // drop lowest set bit: go to block before this one
  for (; i > 0; i -= i & -i) sum += bit_arr[i];
  return sum;
}
```

## LazySegTree.h

**Description:** Segment tree with lazy propagation: range add, range sum, *0-indexed inclusive* $[l, r]$. Use when both updates and queries are on ranges; change the combine in `build/update/seg_sum` and the tag logic in `push` for min/max/assign.
**How it works:** node $v$ stores the sum of a segment; its children are the two halves. A range operation only touches $O(\log n)$ nodes that lie fully inside the range. Instead of updating all elements below such a node, the amount is parked in `lazy[v]` and handed to the children (`push`) only when something later needs to go deeper.
**Modify / tips:** range add + range *min*: in `build`, `update`, `seg_sum` replace `+` of the two children by `min`, return `LLONG_MAX` for an empty range, and add `lazy` to `t` *without* multiplying by the length. Range *assign*: keep a second array saying "lazy is set", and overwrite (`=`) instead of `+=`.
**Usage:** `buildSegTree(a); seg_add(l, r, val); query_sum(l, r);`
**Time:** $O(n)$ build, $O(\log n)$ per operation

67 lines

```
// node v covers a[tl..tr], root v = 1, children 2v, 2v+1
// t[v] = sum of the node, lazy[v] = add owed to children
int seg_n;
vector<int> t, lazy;
// internal: fill t[] for node v from a[tl..tr]
void build(int v, int tl, int tr, const vector<int>& a) {
  if (tl == tr) {
    t[v] = a[tl];
  } else {
    int tm = (tl + tr) / 2;
    build(v * 2, tl, tm, a);
    build(v * 2 + 1, tm + 1, tr, a);
    t[v] = t[v * 2] + t[v * 2 + 1];
  }
}
void push(int v, int tl, int tr) { // give lazy[v] to children
  if (lazy[v]) {
    int tm = (tl + tr) / 2;
    t[v * 2] += lazy[v] * (tm - tl + 1);
    lazy[v * 2] += lazy[v];
    t[v * 2 + 1] += lazy[v] * (tr - tm);
    lazy[v * 2 + 1] += lazy[v];
    lazy[v] = 0;
  }
}
// internal: a[i] += addend for all l <= i <= r
void update(int v, int tl, int tr, int l, int r, int addend) {
  if (l > r)
    return;
  if (l == tl && r == tr) { // node fully inside: stop here
    t[v] += addend * (tr - tl + 1);
    lazy[v] += addend;
  } else {
    push(v, tl, tr);
    int tm = (tl + tr) / 2;
    update(v * 2, tl, tm, l, min(r, tm), addend);
    update(v * 2 + 1, tm + 1, tr, max(l, tm + 1), r, addend);
    t[v] = t[v * 2] + t[v * 2 + 1];
  }
}
// internal: sum of a[l..r] inside node v
int seg_sum(int v, int tl, int tr, int l, int r) {
  if (l > r)  return 0;
  if (l == tl && r == tr) {
    return t[v];
  }
  push(v, tl, tr);
  int tm = (tl + tr) / 2;
  return seg_sum(v * 2, tl, tm, l, min(r, tm))
    + seg_sum(v * 2 + 1, tm + 1, tr, max(l, tm + 1), r);
}
// CALL ONCE: sizes t, lazy to 4 * seg_n, builds from a
void buildSegTree(const vector<int>& a) {
  seg_n = a.size();
  t.assign(4 * seg_n, 0);
  lazy.assign(4 * seg_n, 0);
  if (seg_n > 0) build(1, 0, seg_n - 1, a);
}
// CALL: a[i] += val for all l <= i <= r (0-indexed)
void seg_add(int l, int r, int val) {
  if (seg_n > 0) update(1, 0, seg_n - 1, l, r, val);
}
// CALL: a[l] + ... + a[r] (0-indexed)
int query_sum(int l, int r) {
  if (seg_n == 0) return 0;
  return seg_sum(1, 0, seg_n - 1, l, r);
}
```

## MinMaxSegTree.h

**Description:** Segment tree for range minimum and maximum where single elements can *change* (0-indexed, inclusive $[l, r]$). Use instead of the sparse table as soon as the problem has updates mixed with queries, e.g. "set $a[i] = x$" and "min / max of $a[l..r]$". `first_at_least` finds the first position from $l$ whose value is $\geq x$ ("first free hotel with enough rooms", "next element that fits").
**How it works:** every node stores the min and max of its segment. An update walks from the root down to one leaf and recomputes the nodes on the way back; a query collects $O(\log n)$ nodes that exactly cover $[l, r]$. The search goes left whenever the left child's max is big enough.
**Modify / tips:** for sum / gcd / xor replace `min` and `LLONG_MAX` by the operation and its neutral value (0). For "add to $a[pos]$" write `+= value` at the leaf. For range updates use the lazy tree instead.
**Usage:** `buildMinMaxTree(a); seg_set(pos, value); range_min(l, r); range_max(l, r); first_at_least(l, x);`
**Time:** $O(n)$ build, $O(\log n)$ per operation

75 lines

```
int mm_n;
vector<int> mm_min, mm_max;
// node v covers a[tl..tr], children 2v and 2v+1
void mm_build(int v, int tl, int tr, const vector<int>& a) {
  if (tl == tr) {
    mm_min[v] = mm_max[v] = a[tl];
  } else {
    int tm = (tl + tr) / 2;
    mm_build(v * 2, tl, tm, a);
    mm_build(v * 2 + 1, tm + 1, tr, a);
    mm_min[v] = min(mm_min[v * 2], mm_min[v * 2 + 1]);
    mm_max[v] = max(mm_max[v * 2], mm_max[v * 2 + 1]);
  }
}
// internal: a[pos] = value
void mm_update(int v, int tl, int tr, int pos, int value) {
  if (tl == tr) {
    mm_min[v] = mm_max[v] = value; // the leaf of a[pos]
  } else {
    int tm = (tl + tr) / 2;
    if (pos <= tm) {
      mm_update(v * 2, tl, tm, pos, value);
    } else {
      mm_update(v * 2 + 1, tm + 1, tr, pos, value);
    }
    mm_min[v] = min(mm_min[v * 2], mm_min[v * 2 + 1]);
    mm_max[v] = max(mm_max[v * 2], mm_max[v * 2 + 1]);
  }
}
int mm_get_min(int v, int tl, int tr, int l, int r) {
  if (l > r) return LLONG_MAX; // empty: neutral for min
  if (l == tl && r == tr) return mm_min[v];
  int tm = (tl + tr) / 2;
  return min(mm_get_min(v * 2, tl, tm, l, min(r, tm)),
    mm_get_min(v * 2 + 1, tm + 1, tr, max(l, tm + 1), r));
}
int mm_get_max(int v, int tl, int tr, int l, int r) {
  if (l > r) return LLONG_MIN; // empty: neutral for max
  if (l == tl && r == tr) return mm_max[v];
  int tm = (tl + tr) / 2;
  return max(mm_get_max(v * 2, tl, tm, l, min(r, tm)),
    mm_get_max(v * 2 + 1, tm + 1, tr, max(l, tm + 1), r));
}
// internal: smallest index >= l with a[index] >= x, or -1
int mm_first(int v, int tl, int tr, int l, int x) {
  if (tr < l || mm_max[v] < x) return -1; // nothing here
  if (tl == tr) return tl;
  int tm = (tl + tr) / 2;
  int left = mm_first(v * 2, tl, tm, l, x);
  if (left != -1) return left;
  return mm_first(v * 2 + 1, tm + 1, tr, l, x);
}
// CALL ONCE
void buildMinMaxTree(const vector<int>& a) {
  mm_n = a.size();
  mm_min.assign(4 * mm_n, LLONG_MAX);
  mm_max.assign(4 * mm_n, LLONG_MIN);
  if (mm_n > 0) mm_build(1, 0, mm_n - 1, a);
}
// CALL: a[pos] = value
void seg_set(int pos, int value) {
  mm_update(1, 0, mm_n - 1, pos, value);
}
// CALL: smallest / largest of a[l..r]
int range_min(int l, int r) {
  return mm_get_min(1, 0, mm_n - 1, l, r);
}
int range_max(int l, int r) {
  return mm_get_max(1, 0, mm_n - 1, l, r);
}
// CALL: first index >= l with a[index] >= x, -1 if none
int first_at_least(int l, int x) {
  if (mm_n == 0) return -1;
```

```
  return mm_first(1, 0, mm_n - 1, l, x);
}
```

## SparseTable.h

**Description:** Sparse table for range min and max on a *static* array, 0-indexed inclusive $[L, R]$. Use for RMQ with no updates, LCA via Euler tour, or any idempotent function (gcd, and, or).
**How it works:** `st_min[i][j]` = minimum of the $2^j$ elements starting at $i$, built from two halves of size $2^{j-1}$. A query $[L, R]$ is covered by two blocks of size $2^j$ (one starting at $L$, one ending at $R$) that may overlap, which is fine for min / max.
**Modify / tips:** works for gcd, AND, OR (overlap does not matter) but *not* for sum. If the array changes, use MinMaxSegTree.
**Usage:** `buildSparseTable(a); query_min(L, R);`
**Time:** $O(n \log n)$ build, $O(1)$ per query

33 lines

```
int st_n, max_log;
vector<vector<int>> st_min, st_max;
vector<int> log_table;
// call once, a must not change
void buildSparseTable(vector<int>& a) {
  st_n = a.size();
  log_table.assign(st_n + 1, 0);
  for (int i = 2; i <= st_n; i++)
    log_table[i] = log_table[i / 2] + 1;
  max_log = log_table[st_n] + 1;
  st_min.assign(st_n, vector<int>(max_log));
  st_max.assign(st_n, vector<int>(max_log));
  for (int i = 0; i < st_n; i++) {
    st_min[i][0] = a[i];
    st_max[i][0] = a[i];
  }
  for (int j = 1; j < max_log; j++) {
    for (int i = 0; i + (1 << j) <= st_n; i++) {
      st_min[i][j] = min(st_min[i][j - 1],
        st_min[i + (1 << (j - 1))][j - 1]);
      st_max[i][j] = max(st_max[i][j - 1],
        st_max[i + (1 << (j - 1))][j - 1]);
    }
  }
}
int query_min(int L, int R) { // min of a[L..R]
  int j = log_table[R - L + 1]; // largest 2^j <= length
  return min(st_min[L][j], st_min[R - (1 << j) + 1][j]);
}
int query_max(int L, int R) { // max of a[L..R]
  int j = log_table[R - L + 1]; // largest 2^j <= length
  return max(st_max[L][j], st_max[R - (1 << j) + 1][j]);
}
```

## MonotonicStack.h

**Description:** Stack that is kept sorted, so every index is pushed and popped once. `next_greater`: for each $i$ the nearest index to the right with a strictly larger value. `largest_rectangle`: biggest rectangle under a histogram. Use for "first bigger / smaller element on the left / right", how far each element stays the minimum or maximum (sum of subarray minimums: count subarrays where $a[i]$ is the minimum using previous-smaller and next-smaller), stock span, largest rectangle of 1s in a 0/1 matrix (per row keep `h[j]` = number of consecutive 1s ending at this row, call `largest_rectangle(h)` on every row).
**How it works:** the stack holds indices that have not found their answer yet; their values are sorted, so when a new element arrives it answers (and pops) everything on top that it beats.
**Time:** $O(n)$

35 lines

```
// result[i] = index of the next element > a[i], -1 if none
vector<int> next_greater(vector<int>& a) {
  int n = a.size();
  vector<int> result(n, -1);
  stack<int> st; // indices still waiting for an answer
  // MODIFY: loop i from n - 1 down to 0 for PREVIOUS greater
  for (int i = 0; i < n; i++) {
    // MODIFY the comparison: <  next greater
    //   <= next greater or equal     >  next smaller
    //   >= next smaller or equal
    while (!st.empty() && a[st.top()] < a[i]) {
      result[st.top()] = i; // a[i] is the answer for the top
      st.pop();
    }
    st.push(i);
  }
  return result;
}
// largest area of a rectangle inside the histogram h
int largest_rectangle(vector<int>& h) {
  int n = h.size(), best = 0;
  stack<int> st; // indices of bars, heights increasing
  for (int i = 0; i <= n; i++) {
    int cur = (i == n) ? 0 : h[i]; // 0 at the end empties st
    while (!st.empty() && h[st.top()] >= cur) {
      int height = h[st.top()]; // this bar is the lowest
      st.pop();
      // it extends from the bar below it on the stack to i
      int left = st.empty() ? -1 : st.top();
      best = max(best, height * (i - left - 1));
    }
    st.push(i);
  }
  return best;
}
```

# Graphs (5)

## BFS.h

**Description:** Breadth-first search from $s$: `d[v]` = fewest edges from $s$ ($-1$ if unreachable), `p[v]` = parent on a shortest path. Use for unweighted shortest paths, path recovery (follow `p`), grid/level problems.
**How it works:** the queue hands out vertices in order of distance from $s$, so the first time a vertex is reached is along a shortest path.
**Modify / tips:** path from $s$ to $t$: start at $t$, follow `p[]` until $-1$, reverse. Grid: a vertex is a cell, neighbours come from `dx`, `dy` (see Multi-SourceBFS). Two-colouring / bipartite check: give $u$ the opposite colour of $v$; a neighbour with the same colour means an odd cycle. Extra condition (keys, remaining jumps): BFS over pairs (vertex, condition).
**Time:** $O(V + E)$

24 lines

```
vector<vector<int>> adj;
vector<int> d, p;
vector<bool> vis;
// d.assign(n + 1, -1);
// p.assign(n + 1, -1);
// vis.assign(n + 1, false);
void bfs(int s) { // d[] = edges from s, p[] = parent
  queue<int> q;
  q.push(s);
  vis[s] = true;
  d[s] = 0;
  while(!q.empty()) {
    int v = q.front();
    q.pop();
    for(int u : adj[v]) {
      if (!vis[u]) { // first visit = shortest
        vis[u] = true;
        d[u] = d[v] + 1;
        p[u] = v;
        q.push(u);
      }
    }
  }
}
```

## DFS.h

**Description:** Depth-first search with entry/exit times. `color`: 0 unvisited, 1 on stack, 2 done. Use for subtree ranges ($u$ is an ancestor of $v$ iff $tin[u] \le tin[v]$ and $tout[v] \le tout[u]$), cycle detection (edge to a colour-1 vertex), topological order (reverse `tout`). The iterative version is safe for depth $> 10^5$.
**How it works:** go as deep as possible, then back up. `tin` / `tout` are clock values on entering and leaving a vertex; everything entered between them is in the subtree. The iterative version pushes each vertex twice: once to enter, once to leave.
**Modify / tips:** cycle in a directed graph: while entering $v$, a neighbour with colour 1. Subtree queries: number vertices by entry order; the subtree of $v$ becomes one contiguous range, so a Fenwick / segment tree can answer "sum in subtree". `timer` here counts entries and exits together ($0..2n-1$). Call once per unvisited vertex if the graph is disconnected (then delete the line `timer = 0;` and reset it yourself once).
**Time:** $O(V + E)$

34 lines

```
vector<int> color, tin, tout;
int timer;
// color.assign(n + 1, 0);
// tin.assign(n + 1, -1);
// tout.assign(n + 1, -1);
void iterative_dfs(int root) { // tin[], tout[], no recursion
  timer = 0;
  stack<pair<int,int>> st;
  st.push({root, 0}); // 0 = enter, 1 = exit
  while (!st.empty()) {
    auto [v, state] = st.top();
    st.pop();
    if (state == 0) {
      if (color[v] != 0) continue; // already entered
      tin[v] = timer++;
      color[v] = 1;
      st.push({v, 1});
      for (int i = (int)adj[v].size() - 1; i >= 0; i--) {
        int u = adj[v][i];
        if (color[u] == 0) st.push({u, 0});
      }
    } else {
      color[v] = 2;
      tout[v] = timer++;
    }
  }
```

```
}
void recursive_dfs(int v) { // marks vis[] reachable from v
  vis[v] = 1;
  for (int u : adj[v]) {
    if (vis[u]) continue;
    recursive_dfs(u);
  }
}
```

## Dijkstra.h

**Description:** Shortest paths from $s$ with *non-negative* edge weights; `adjd[v]` holds {to, weight}. Unreachable vertices keep `LLONG_MAX`. Use for weighted shortest paths; run on a state graph (vertex, extra) for constrained variants.
**How it works:** repeatedly take the unfinished vertex with the smallest known distance (top of the min-heap) and try to improve its neighbours through it. A vertex can sit in the heap several times; outdated copies are skipped.
**Modify / tips:** add edges with `adjd[u].push_back({v, w})` (both directions if undirected). Path: store `p[u] = v` where `dist[u]` improves. Extra resource (one free ride, $k$ coupons): make the vertex number `v * (k + 1) + used` and add the extra transitions. Minimise the largest edge instead of the sum: replace `d + w` by `max(d, w)`.
**Time:** $O((V + E) \log V)$

20 lines

```
vector<vector<pair<int,int>>> adjd;
vector<int> dist;
// dist.assign(n + 1, LLONG_MAX);
void dijkstra(int s) { // dist[] from s, weights >= 0
  priority_queue<pair<int,int>, vector<pair<int,int>>,
    greater<pair<int,int>>> pq;
  dist[s] = 0;
  pq.push({0, s});
  while(!pq.empty()) {
    auto [d, v] = pq.top();
    pq.pop();
    if(d != dist[v]) continue; // outdated copy
    for(auto [u, w] : adjd[v]) {
      if(dist[u] > d + w) { // shorter way to u via v
        dist[u] = d + w;
        pq.push({dist[u], u});
      }
    }
  }
}
```

## BFS01.h

**Description:** Dijkstra specialised to edge weights 0 or 1 using a deque (0-edges go to the front, 1-edges to the back). Use for "minimum number of paid moves / flips / walls broken" problems.
**How it works:** a 0-edge keeps the distance equal, so its end goes to the *front* of the deque; a 1-edge goes to the back. The deque therefore stays sorted by distance without a heap.
**Time:** $O(V + E)$

24 lines

```
// dist.assign(n + 1, LLONG_MAX);
// p.assign(n + 1, -1);
// basically dijkstra but optimized because we have
// weights only as 0-1
void bfs01(int s) { // dist[] from s, p[] = parent
  deque<int> q;
  dist[s] = 0;
  q.push_front(s);
  while(!q.empty()) {
    int v = q.front();
    q.pop_front();
    for(auto [u, w] : adjd[v]) {
      if(dist[v] + w < dist[u]) {
        dist[u] = dist[v] + w;
        p[u] = v;
        if(w == 1) { // cost 1: back, cost 0: front
          q.push_back(u);
        } else {
          q.push_front(u);
        }
      }
    }
  }
}
```

## BellmanFord.h

**Description:** Shortest paths from $s$ with *negative* edges allowed, vertices $1..n$, `adjbf[u]` holds {to, weight}. After it, run the commented loop: any edge that still relaxes means a negative cycle reachable from $s$. Use for negative weights, arbitrage, systems of difference constraints.
**How it works:** a shortest path uses at most $n - 1$ edges, so relaxing every edge $n-1$ times is enough. If something can still be improved after that, a negative cycle exists.
**Modify / tips:** longest path: negate the weights. Difference constraints $x_v - x_u \le w$: add edge $u \to v$ with weight $w$.
**Time:** $O(VE)$

30 lines

```
vector<vector<pair<int,int>>> adjbf;
vector<int> bellman_dist;
// adjbf.assign(n + 1, {});
// bellman_dist.assign(n + 1, LLONG_MAX);
void bellmanFord(int n, int s) { // negative edges allowed
  bellman_dist.assign(n + 1, LLONG_MAX);
  bellman_dist[s] = 0;
  for (int i = 1; i <= n - 1; i++) { // n - 1 rounds
    for(int u = 1; u <= n; u++) {
      if (bellman_dist[u] == LLONG_MAX) continue;
      for (auto [v, w] : adjbf[u]) {
        if (bellman_dist[v] > bellman_dist[u] + w) {
          bellman_dist[v] = bellman_dist[u] + w;
        }
      }
    }
  }
}
// for (int u = 1; u <= n; u++)
// {
//     if (bellman_dist[u] == LLONG_MAX) continue;
//     for (auto [v, w] : adjbf[u])
//     {
//         if (bellman_dist[v] > bellman_dist[u] + w)
//         {
//             // Negative cycle detected.
//             // Handle according to the problem.
//         }
//     }
// }
```

## Prim.h

**Description:** Minimum spanning tree (Prim) of an undirected weighted graph in `adjd` (add every edge in both directions). Returns the total weight, or `LLONG_MAX` if the graph is not connected; `mst_parent[v]` = the vertex $v$ was attached to. Use for "cheapest way to connect everything", minimising the largest edge on paths (the MST also minimises the maximum edge). For a maximum spanning tree negate all weights. Alternative (Kruskal): sort edges by weight and `union_sets` the ends of every edge whose ends are in different sets.
**How it works:** grow one tree from vertex 1; the heap holds every edge leaving the tree and the cheapest one that reaches a new vertex is always taken.
**Time:** $O(E \log V)$

27 lines

```
vector<int> mst_parent;
int prim(int n) { // vertices 1..n
  vector<bool> in_tree(n + 1, false);
  mst_parent.assign(n + 1, -1);
  // {edge weight, {vertex, attached to}}
  priority_queue<pair<int, pair<int,int>>,
    vector<pair<int, pair<int,int>>>,
    greater<pair<int, pair<int,int>>>> pq;
  pq.push({0, {1, -1}}); // start from vertex 1
  int total = 0, taken = 0;
  while (!pq.empty()) {
    int w = pq.top().first;
    int v = pq.top().second.first;
    int from = pq.top().second.second;
    pq.pop();
    if (in_tree[v]) continue;
    in_tree[v] = true; // cheapest edge leaving the tree
    mst_parent[v] = from;
    total += w;
    taken++;
    for (auto [u, weight_u] : adjd[v]) {
      if (!in_tree[u]) pq.push({weight_u, {u, v}});
    }
  }
  if (taken < n) return LLONG_MAX; // not connected
  return total;
}
```

## Components.h

**Description:** Connected components ("islands") by BFS. `components` is for a graph in `adj`; `islands` is for a grid where equal cells touching by a side belong together. Use to count / list isolated groups, sizes of regions, flood fill. For 8 directions extend `dx`, `dy` with the 4 diagonals.
**Time:** $O(V + E)$ graph, $O(nm)$ grid

56 lines

```
// groups[k] = all vertices of the k-th component
// number of components = groups.size()
vector<vector<int>> components(int n) { // vertices 1..n
  vector<vector<int>> groups;
  vector<bool> seen(n + 1, false);
  for (int s = 1; s <= n; s++) {
    if (seen[s]) continue;
    groups.push_back({});
    queue<int> q;
    q.push(s);
    seen[s] = true;
    while (!q.empty()) {
      int v = q.front();
```

```
      q.pop();
      groups.back().push_back(v);
      for (int u : adj[v]) {
        if (!seen[u]) {
          seen[u] = true;
          q.push(u);
        }
      }
    }
  }
  return groups;
}
// island_id[i][j] = which island cell (i, j) is in,
// -1 if the cell is not land. Returns number of islands.
vector<vector<int>> island_id;
int islands(vector<string>& grid, char land) {
  int n = grid.size(), m = grid[0].size(), total = 0;
  int dx[4] = {1, -1, 0, 0}, dy[4] = {0, 0, 1, -1};
  island_id.assign(n, vector<int>(m, -1));
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
      if (grid[i][j] != land) continue;
      if (island_id[i][j] != -1) continue;
      queue<pair<int,int>> q;
      q.push({i, j});
      island_id[i][j] = total;
      while (!q.empty()) {
        auto [x, y] = q.front();
        q.pop();
        for (int k = 0; k < 4; k++) {
          int nx = x + dx[k], ny = y + dy[k];
          if (nx < 0 || ny < 0 || nx >= n || ny >= m) continue;
          if (grid[nx][ny] != land) continue;
          if (island_id[nx][ny] != -1) continue;
          island_id[nx][ny] = total;
          q.push({nx, ny});
        }
      }
      total++;
    }
  }
  return total;
}
```

## MultiSourceBFS.h

**Description:** BFS started from *many* sources at once: every cell / vertex gets the distance to its *nearest* source. Use for "distance to the closest X for every cell", "fire / infection / rot spreads from several places, when is everything reached" (answer = largest distance), "is the exit reached before the fire" (run it once per kind of source and compare). Lines marked MODIFY are the only ones that depend on the problem.
**Time:** $O(nm)$ grid, $O(V+E)$ graph

48 lines

```
vector<vector<int>> dist_grid; // -1 = never reached
void multi_source_bfs(vector<string>& grid) {
  int n = grid.size(), m = grid[0].size();
  // MODIFY: allowed moves (add 4 diagonals for 8 directions)
  int dx[4] = {1, -1, 0, 0}, dy[4] = {0, 0, 1, -1};
  dist_grid.assign(n, vector<int>(m, -1));
  queue<pair<int,int>> q;
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
      if (grid[i][j] == 'S') { // MODIFY: which cells start
        dist_grid[i][j] = 0;
        q.push({i, j});
      }
    }
  }
  while (!q.empty()) {
    auto [x, y] = q.front();
    q.pop();
    for (int k = 0; k < 4; k++) {
      int nx = x + dx[k], ny = y + dy[k];
      if (nx < 0 || ny < 0 || nx >= n || ny >= m) continue;
      if (grid[nx][ny] == '#') continue; // MODIFY: blocked
      if (dist_grid[nx][ny] != -1) continue; // seen before
      dist_grid[nx][ny] = dist_grid[x][y] + 1;
      // MODIFY: to know WHICH source is nearest, also copy
      // owner[nx][ny] = owner[x][y] here
      q.push({nx, ny});
    }
  }
}
// same idea on a graph in adj: d[v] = edges to nearest source
// d.assign(n + 1, -1);
void multi_source_bfs_graph(vector<int>& sources) {
  queue<int> q;
  for (int s : sources) { // MODIFY: the starting vertices
    d[s] = 0;
    q.push(s);
  }
  while (!q.empty()) {
    int v = q.front();
    q.pop();
    for (int u : adj[v]) {
      if (d[u] != -1) continue;
      d[u] = d[v] + 1;
      q.push(u);
    }
  }
}
```

## TopoSort.h

**Description:** Topological order of a *directed* graph in `adj` (Kahn's algorithm): every edge $u \to v$ has $u$ before $v$. If the result has fewer than $n$ vertices the graph has a cycle. Use for dependency ordering, DP on a DAG (process vertices in this order), cycle detection in directed graphs. Swap the queue for a min-heap to get the lexicographically smallest order.
**How it works:** a vertex with no incoming edges can go first; remove it, which may free its neighbours.
**Modify / tips:** DP on a DAG: walk the returned order and push values along edges, `dp[u] = max(dp[u], dp[v] + 1)` for the longest path, `dp[u] += dp[v]` to count paths.
**Time:** $O(V+E)$

20 lines

```
vector<int> topo_sort(int n) { // vertices 1..n
  vector<int> indegree(n + 1, 0), order;
  for (int v = 1; v <= n; v++) {
    for (int u : adj[v]) indegree[u]++;
  }
  queue<int> q;
  for (int v = 1; v <= n; v++) {
    if (indegree[v] == 0) q.push(v);
  }
  while (!q.empty()) {
    int v = q.front();
    q.pop();
    order.push_back(v);
    for (int u : adj[v]) {
      indegree[u]--;
      if (indegree[u] == 0) q.push(u);
    }
  }
  return order; // order.size() < n means there is a cycle
}
```

## TarjanSCC.h

**Description:** Strongly connected components (Tarjan) of a directed graph in `adj`, vertices $1..n$, in one DFS and without a reversed graph. `scc_root[v]` = leader vertex of the component of $v$ (NOT $0..k-1$); $u$ and $v$ reach each other iff `scc_root[u] == scc_root[v]`. `scc_comps[k]` = vertices of the $k$-th component found. Components come out in REVERSE topological order (sinks first). `scc_adj` = condensed graph on leaders (duplicate edges possible). Use for "mutually reachable" groups, shrinking a graph to a DAG, 2-SAT.
**How it works:** `scc_in[u]` = DFS entry time. `scc_low[u]` = smallest entry time reachable from $u$'s subtree using one edge to a vertex still on the stack. Vertices stay on `scc_stk` until their component is complete. When `scc_low[u] == scc_in[u]`, nothing below $u$ reaches above $u$, so $u$ and everything above it on the stack form one component: pop them.
**Modify / tips:** DP on the condensed DAG: walk `scc_comps` from first to last, every component reachable from the current one is already done. Remove duplicate edges of `scc_adj` with `sort` + `unique` if needed. Number of components = `scc_comps.size()`. `scc_dfs` is recursive, very deep graphs may overflow the stack.
**Usage:** `scc_root, scc_in, scc_low .assign(n + 1, -1); scc_adj.assign(n + 1, {}); tarjan(n);`
**Time:** $O(V+E)$

39 lines

```
vector<int> scc_stk, scc_root, scc_in, scc_low;
vector<vector<int>> scc_comps, scc_adj;
int scc_timer;
void scc_dfs(int u) { // enter u, push on stack
    scc_low[u] = scc_in[u] = scc_timer++;
    scc_stk.push_back(u);
    for (int v : adj[u]) {
        if (scc_in[v] == -1) { // tree edge
            scc_dfs(v);
            scc_low[u] = min(scc_low[u], scc_low[v]);
        } else if (scc_root[v] == -1) { // v still on stack
            scc_low[u] = min(scc_low[u], scc_in[v]);
        }
    }
    if (scc_low[u] == scc_in[u]) { // u is a component root
        scc_comps.push_back({u});
        while (true) {
            int v = scc_stk.back();
            scc_stk.pop_back();
            scc_root[v] = u;
            if (v == u) break;
            scc_comps.back().push_back(v);
        }
    }
}
void tarjan(int n) { // vertices 1..n
    scc_stk.clear();
    scc_comps.clear();
```

```
    scc_timer = 0;
    for (int u = 1; u <= n; u++) {
        if (scc_in[u] == -1) scc_dfs(u);
    }
    for (int u = 1; u <= n; u++) { // condensed graph
        for (int v : adj[u]) {
            if (scc_root[u] != scc_root[v])
                scc_adj[scc_root[u]].push_back(scc_root[v]);
        }
    }
}
```

## LCA.h

**Description:** Lowest common ancestor by binary lifting on a tree in `adj`. `lca(u, v)` = deepest vertex that is an ancestor of both; `tree_dist` = number of edges on the path. Use for distance / path queries between two vertices, checking if a vertex lies on a path, $k$-th ancestor (jump with `up`). Built with BFS, so deep trees are safe.
**How it works:** `up[v][j]` is the ancestor $2^j$ steps above $v$ (the root's ancestor is itself). To answer: lift the deeper vertex to the same depth, then lift both by the largest jumps that keep them different; their parent is the answer.
**Modify / tips:** $k$-th ancestor: jump by the 1-bits of $k$. Weighted distance: keep `dist_root[v]` and use it in place of `depth` in `tree_dist`. Max edge on a path: store `mx[v][j]` next to `up[v][j]` and combine it on every jump. Is $x$ on the path $u..v$: `tree_dist(u,x) + tree_dist(x,v) == tree_dist(u,v)`.
**Usage:** `build_lca(root, n); lca(u, v); tree_dist(u, v);`
**Time:** $O(n \log n)$ build, $O(\log n)$ per query

40 lines

```
const int LOG = 20; // need 2^LOG > n
vector<vector<int>> up; // up[v][j] = 2^j-th ancestor of v
vector<int> depth;      // edges from root to v
void build_lca(int root, int n) { // vertices 1..n
  up.assign(n + 1, vector<int>(LOG, root));
  depth.assign(n + 1, 0);
  queue<int> q;
  q.push(root);
  while (!q.empty()) {
    int v = q.front();
    q.pop();
    for (int j = 1; j < LOG; j++) {
      up[v][j] = up[up[v][j - 1]][j - 1];
    }
    for (int u : adj[v]) {
      if (u == up[v][0]) continue; // skip the parent
      up[u][0] = v;
      depth[u] = depth[v] + 1;
      q.push(u);
    }
  }
}
int lca(int u, int v) {
  if (depth[u] < depth[v]) swap(u, v);
  int diff = depth[u] - depth[v];
  for (int j = 0; j < LOG; j++) { // lift u to depth of v
    if ((diff >> j) & 1) u = up[u][j];
  }
  if (u == v) return u;
  for (int j = LOG - 1; j >= 0; j--) { // lift both below lca
    if (up[u][j] != up[v][j]) {
      u = up[u][j];
      v = up[v][j];
    }
  }
  return up[u][0];
}
int tree_dist(int u, int v) { // edges between u and v
  return depth[u] + depth[v] - 2 * depth[lca(u, v)];
}
```

## Bipartite.h

**Description:** Two-colouring check of an undirected graph in `adj` (vertices $1..n$, every edge stored in both directions). Returns true iff the graph has no odd cycle; `side[v]` in $\{0,1\}$ is then a valid colouring, i.e. the two sides. Works for disconnected graphs. Use for "split into two groups with no conflict inside a group", odd-cycle detection, setting up bipartite matching.
**How it works:** colour a start vertex 0 and run BFS; every neighbour must get the opposite colour. A neighbour that already has the same colour closes an odd cycle. A graph is bipartite iff it has no odd cycle.
**Modify / tips:** a self-loop makes the answer false. If the graph is bipartite, the number of valid 2-colourings is `2^(number of components)`. On a grid the colouring is simply `(i + j) % 2`, no search needed.
**Usage:** `side.assign(n + 1, -1); bool ok = bipartite(n);`
**Time:** $O(V + E)$

23 lines

```
vector<int> side; // 0 / 1 colour, -1 = not coloured yet
bool bipartite(int n) { // vertices 1..n
    bool ok = true;
    for (int s = 1; s <= n; s++) {
        if (side[s] != -1) continue;
        queue<int> q;
        q.push(s);
        side[s] = 0;
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            for (int u : adj[v]) {
                if (side[u] == -1) {
                    side[u] = side[v] ^ 1; // opposite colour
                    q.push(u);
                } else if (side[u] == side[v]) {
                    ok = false; // odd cycle
                }
            }
        }
    }
    return ok;
}
```

# Dynamic programming (6)

## LCS.h

**Description:** Longest common subsequence of two strings, returned as the actual string (its `.size()` is the length). `dp[i][j]` = LCS length of the first $i$ characters of $a$ and first $j$ of $b$. Use for edit-style problems: minimum deletions to make strings equal $= n + m - 2 \cdot LCS$, shortest common supersequence, longest palindromic subsequence (LCS of $s$ and reversed $s$). Memory is $8nm$ bytes, fine up to about $n, m \leq 3000$.
**How it works:** if the last characters match they are paired (diagonal $+1$); otherwise drop the last character of one string, whichever gives more. The answer is rebuilt by walking the table backwards.
**Modify / tips:** edit distance: `dp[i][0] = i, dp[0][j] = j`, equal characters copy the diagonal, otherwise `1 + min(up, left, diagonal)`. Works on `vector<int>` the same way.
**Time:** $O(nm)$

28 lines

```
string lcs(string a, string b) {
  int n = a.size(), m = b.size();
  vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
  for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
      if (a[i - 1] == b[j - 1]) {
        dp[i][j] = dp[i - 1][j - 1] + 1;
      } else {
        dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
      }
    }
  }
  string result; // walk back from dp[n][m]
  int i = n, j = m;
  while (i > 0 && j > 0) {
    if (a[i - 1] == b[j - 1]) {
      result.push_back(a[i - 1]);
      i--;
      j--;
    } else if (dp[i - 1][j] >= dp[i][j - 1]) {
      i--;
    } else {
      j--;
    }
  }
  reverse(result.begin(), result.end());
  return result;
}
```

## LIS.h

**Description:** Longest *strictly* increasing subsequence with binary search, returned as the actual values (its `.size()` is the length). `tail[k]` = smallest possible last value of an increasing subsequence of length $k + 1$. For non-decreasing (equal values allowed) change `lower_bound` to `upper_bound`. For longest decreasing, negate or reverse the array first. Use for nested envelopes / boxes (sort by one side first), minimum number of decreasing chains, patience-sort style problems.
**How it works:** go left to right; each element replaces the first tail that is $\geq$ it (found by binary search) or extends the list. `previous[]` remembers what came before each element so the sequence can be rebuilt.
**Time:** $O(n \log n)$

23 lines

```
vector<int> lis(vector<int>& a) {
  int n = a.size();
  vector<int> tail, tail_index, previous(n, -1);
  for (int i = 0; i < n; i++) {
    int pos = lower_bound(tail.begin(), tail.end(), a[i])
      - tail.begin();
    if (pos == (int)tail.size()) {
      tail.push_back(a[i]);
      tail_index.push_back(i);
    } else {
      tail[pos] = a[i];
      tail_index[pos] = i;
    }
    if (pos > 0) previous[i] = tail_index[pos - 1];
  }
```

```
  vector<int> result; // rebuild from the last element
  if (n == 0) return result;
  for (int i = tail_index.back(); i != -1; i = previous[i]) {
    result.push_back(a[i]);
  }
  reverse(result.begin(), result.end());
  return result;
}
```

## Knapsack.h

**Description:** All knapsack variants. `dp[w]` = best answer using total weight (or sum) exactly / at most $w$. **Which one:** each item usable once → 0/1 (inner loop goes *down*); unlimited copies → unbounded (inner loop goes *up*); limited copies → bounded; only "is it possible" → subset sum; "how many ways" → counting; "fewest items" → min coins; capacity up to $10^9$ but values small → by value. For "weight *exactly* $W$" in (1)–(3) start with `dp[0] = 0` and all others `LLONG_MIN` and skip those. Splitting an array into two parts with minimal difference = subset sum on half the total.
**Time:** $O(nW)$ each; bounded $O(W \sum \log cnt)$; by value $O(n \sum val)$

110 lines

```
// (1) 0/1: every item at most once, max value, weight <= W
int knapsack_01(vector<int>& wt, vector<int>& val, int W) {
  int n = wt.size();
  vector<int> dp(W + 1, 0);
  for (int i = 0; i < n; i++) {
    for (int w = W; w >= wt[i]; w--) { // DOWN: used once
      dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
    }
  }
  return dp[W];
}
// (2) unbounded: every item any number of times
int knapsack_unbounded(vector<int>& wt, vector<int>& val,
    int W) {
  int n = wt.size();
  vector<int> dp(W + 1, 0);
  for (int i = 0; i < n; i++) {
    for (int w = wt[i]; w <= W; w++) { // UP: reused
      dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
    }
  }
  return dp[W];
}
// (3) bounded: item i at most cnt[i] times. Split cnt[i]
// into packs of 1, 2, 4, ... and treat each pack as 0/1
int knapsack_bounded(vector<int>& wt, vector<int>& val,
    vector<int>& cnt, int W) {
  int n = wt.size();
  vector<int> dp(W + 1, 0);
  for (int i = 0; i < n; i++) {
    int left = cnt[i];
    for (int pack = 1; left > 0; pack *= 2) {
      int take = min(pack, left);
      left -= take;
      for (int w = W; w >= take * wt[i]; w--) {
        dp[w] = max(dp[w],
          dp[w - take * wt[i]] + take * val[i]);
      }
    }
  }
  return dp[W];
}
// (4) subset sum: can some items add up to exactly S
// faster: bitset<100001> b; b[0] = 1; b |= b << x; per x
bool subset_sum(vector<int>& a, int S) {
  vector<bool> dp(S + 1, false);
  dp[0] = true;
  for (int x : a) {
    for (int s = S; s >= x; s--) {
      if (dp[s - x]) dp[s] = true;
    }
  }
  return dp[S];
}
// (5a) number of subsets with sum exactly S (each item once)
int count_subsets(vector<int>& a, int S) {
  vector<int> dp(S + 1, 0);
  dp[0] = 1;
  for (int x : a) {
    for (int s = S; s >= x; s--) {
      dp[s] = (dp[s] + dp[s - x]) % MOD;
    }
  }
  return dp[S];
}
// (5b) ways to make S with unlimited coins, order ignored
// (1+2 and 2+1 are the same). If order MATTERS, swap the
// two loops: sum outside, coins inside
int coin_ways(vector<int>& coins, int S) {
  vector<int> dp(S + 1, 0);
  dp[0] = 1;
  for (int c : coins) {
    for (int s = c; s <= S; s++) {
      dp[s] = (dp[s] + dp[s - c]) % MOD;
    }
  }
  return dp[S];
}
// (6) fewest coins to make exactly S, -1 if impossible
int min_coins(vector<int>& coins, int S) {
  vector<int> dp(S + 1, LLONG_MAX);
  dp[0] = 0;
  for (int c : coins) {
    for (int s = c; s <= S; s++) { // DOWN if each coin once
      if (dp[s - c] == LLONG_MAX) continue;
      dp[s] = min(dp[s], dp[s - c] + 1);
    }
  }
  return dp[S] == LLONG_MAX ? -1 : dp[S];
}
// (7) 0/1 when W is huge but values are small:
// dp[v] = smallest weight that gives value exactly v
int knapsack_by_value(vector<int>& wt, vector<int>& val,
    int W) {
  int n = wt.size(), total = 0;
  for (int v : val) total += v;
  vector<int> dp(total + 1, LLONG_MAX);
  dp[0] = 0;
  for (int i = 0; i < n; i++) {
    for (int v = total; v >= val[i]; v--) {
      if (dp[v - val[i]] == LLONG_MAX) continue;
      dp[v] = min(dp[v], dp[v - val[i]] + wt[i]);
    }
  }
  int best = 0;
  for (int v = 0; v <= total; v++) {
    if (dp[v] <= W) best = v;
  }
  return best;
}
```

## BitmaskDP.h

**Description:** DP over subsets, for $n \leq 20$ (the hint is a tiny $n$). Bit $i$ of `mask` = item $i$ already used. Two standard shapes: `dp[mask][last]` when the *order* matters and the cost depends on the previous item (travelling salesman, shortest Hamiltonian path, ordering jobs); `dp[mask]` when items are matched one by one (assign $n$ people to $n$ jobs, pairing). To count instead of minimise: start with 1, replace `min` by `+` mod. Lines marked MODIFY depend on the problem.
**Time:** $O(2^n n^2)$ tsp, $O(2^n n)$ assignment

47 lines

```
// min cost to start at 0, visit every vertex once, return
int tsp(vector<vector<int>>& cost) {
  int n = cost.size(), FULL = (1LL << n);
  // dp[mask][last] = cheapest way to have visited the set
  // mask and be standing at last
  vector<vector<int>> dp(FULL, vector<int>(n, LLONG_MAX));
  dp[1][0] = 0; // MODIFY: start (any start: dp[1 << i][i] = 0)
  for (int mask = 1; mask < FULL; mask++) {
    for (int last = 0; last < n; last++) {
      if (dp[mask][last] == LLONG_MAX) continue;
      for (int nxt = 0; nxt < n; nxt++) {
        if ((mask >> nxt) & 1) continue; // already visited
        int new_mask = mask | (1LL << nxt);
        // MODIFY: cost of going from last to nxt
        dp[new_mask][nxt] = min(dp[new_mask][nxt],
          dp[mask][last] + cost[last][nxt]);
      }
    }
  }
  int best = LLONG_MAX;
  for (int last = 0; last < n; last++) {
    if (dp[FULL - 1][last] == LLONG_MAX) continue;
    // MODIFY: remove "+ cost[last][0]" if no return needed
    best = min(best, dp[FULL - 1][last] + cost[last][0]);
  }
  return best;
}
// min total cost giving each person a different job
int assignment(vector<vector<int>>& cost) {
  int n = cost.size(), FULL = (1LL << n);
  // dp[mask] = best cost when the first popcount(mask)
  // people have taken exactly the jobs in mask
  vector<int> dp(FULL, LLONG_MAX);
  dp[0] = 0;
  for (int mask = 0; mask < FULL - 1; mask++) {
    if (dp[mask] == LLONG_MAX) continue;
    int person = __builtin_popcountll(mask);
    for (int job = 0; job < n; job++) {
      if ((mask >> job) & 1) continue; // job already taken
      int new_mask = mask | (1LL << job);
      // MODIFY: cost of giving this job to this person
      dp[new_mask] = min(dp[new_mask],
        dp[mask] + cost[person][job]);
    }
  }
  return dp[FULL - 1];
```

```
}
```

## DigitDP.h

**Description:** Count the numbers in $[L, R]$ whose *digits* satisfy a property, for $R$ up to $10^{18}$ (the hint is a huge range plus a condition on digits). Build the number digit by digit from the left. `pos` = digit being placed; `state` = the only thing you need to remember about the digits so far; `tight` = digits so far equal the prefix of $N$, so this digit cannot exceed $N$'s digit; `started` = a non-zero digit was already placed (false = still leading zeros). The example counts numbers whose digit sum is divisible by `digit_k`. Other states: last digit (no two equal neighbours), count of some digit, number mod $m$ (`state * 10 + digit`) % $m$, mask of used digits.
**Usage:** `digit_k = 7; int ans = count_upto(R) - count_upto(L - 1);`
**Time:** $O(\text{digits} \cdot \text{states} \cdot 10)$

36 lines

```
string digits; // decimal digits of N
int digit_k;   // MODIFY: parameters of the property
vector<vector<vector<int>>> memo; // [pos][state][started]
int digit_dp(int pos, int state, bool tight, bool started) {
  if (pos == (int)digits.size()) {
    // MODIFY: is the finished number good? (started is
    // false only for the number 0)
    return (started && state == 0) ? 1 : 0;
  }
  if (!tight && memo[pos][state][started] != -1) {
    return memo[pos][state][started];
  }
  int limit = tight ? digits[pos] - '0' : 9;
  int result = 0;
  for (int digit = 0; digit <= limit; digit++) {
    bool new_started = started || digit != 0;
    int new_state = state;
    if (new_started) {
      // MODIFY: how one more digit changes the state
      new_state = (state + digit) % digit_k;
    }
    // MODIFY: "continue;" here to forbid this digit
    result += digit_dp(pos + 1, new_state,
      tight && digit == limit, new_started);
  }
  if (!tight) memo[pos][state][started] = result;
  return result;
}
int count_upto(int N) { // good numbers in [1, N]
  if (N <= 0) return 0;
  digits = to_string(N);
  // MODIFY: digit_k = number of different states
  memo.assign(digits.size(), vector<vector<int>>(digit_k,
    vector<int>(2, -1)));
  return digit_dp(0, 0, true, false);
}
```

## TreeDP.h

**Description:** DP on subtrees skeleton. Recipe: (1) decide what `dp[v]` means for the subtree of $v$ only, with one array per choice made at $v$; (2) write the value for $v$ alone; (3) after solving a child $u$, merge its answer into $v$; (4) read the answer at the root. The example is maximum total weight of vertices with no two adjacent (`dp_take`: $v$ chosen, `dp_skip`: $v$ not chosen). To adapt, change only the numbered lines. Common variants: count instead of max (use + / × mod), `subtree[v]` for sizes, longest path through $v$ = two largest child depths. Tree in `adj`, undirected; recursion depth is the tree height.
**Time:** $O(n)$

18 lines

```
vector<int> weight, subtree, dp_take, dp_skip;
// subtree.assign(n + 1, 0);
// dp_take.assign(n + 1, 0);
// dp_skip.assign(n + 1, 0);
// tree_dp(root, 0);
void tree_dp(int v, int par) {
  subtree[v] = 1;              // (2) v alone
  dp_take[v] = weight[v];
  dp_skip[v] = 0;
  for (int u : adj[v]) {
    if (u == par) continue;
    tree_dp(u, v);             // solve the child first
    subtree[v] += subtree[u]; // (3) merge child u into v
    dp_take[v] += dp_skip[u];
    dp_skip[v] += max(dp_take[u], dp_skip[u]);
  }
}
// (4) answer = max(dp_take[root], dp_skip[root])
```

# Strings (7)

## KMP.h

**Description:** `pi[i]` = length of the longest proper prefix of $s[0..i]$ that is also its suffix. `kmp` returns all 0-indexed start positions of `substr` in `parent`. Use for pattern matching, borders, smallest period $(n - pi[n-1]$ if it divides $n)$.
**How it works:** `j` = how many characters of the pattern currently match. On a mismatch, instead of restarting, drop to the next shorter prefix that is still a suffix of what was matched (`pi[j - 1]`).
**Modify / tips:** number of occurrences = `matches.size()`. Longest prefix of $s$ that is also a suffix = `pi[n-1]`. Works on vectors too.
**Time:** $O(|substr| + |parent|)$

33 lines

```
// pi[i] = border of s[0..i]
vector<int> prefix_function(string s) {
  int n = s.length();
  vector<int> pi(n, 0);
  for (int i = 1; i < n; i++) {
    int j = pi[i - 1];
    while (j > 0 && s[i] != s[j]) { // try shorter border
      j = pi[j - 1];
    }
    if (s[i] == s[j]) j++;
    pi[i] = j;
  }
  return pi;
}
// starts of substr in parent
vector<int> kmp(string substr, string parent) {
  vector<int> matches;
  if (substr.empty() || parent.empty()
    || substr.length() > parent.length()) return matches;
  vector<int> pi = prefix_function(substr);
  int j = 0;
  for (int i = 0; i < parent.length(); i++) {
    while (j > 0 && parent[i] != substr[j]) {
      j = pi[j - 1];
    }
    if (parent[i] == substr[j]) j++;
    if (j == substr.length()) { // whole pattern matched
      matches.push_back(i - j + 1);
      j = pi[j - 1];
    }
  }
  return matches;
}
```

## Manacher.h

**Description:** Longest palindrome around every centre in $O(n)$. Works on `t` = `s` with `#` between and around the characters and sentinels `^` `$` at the ends (`t` has $2|s| + 3$ characters). `p[i]` = length of the longest palindrome of `s` centred at `t[i]`: odd length if `t[i]` is a letter, even length if `t[i]` is `#`. It starts at index `(i - p[i]) / 2` of `s`. Use for longest palindromic substring, counting palindromic substrings, and $O(1)$ "is `s[l..r]` a palindrome" queries.
**How it works:** keep the palindrome that reaches furthest right (centre $C$, right end $R$). For a new centre $i$ inside it, the mirror position $2C - i$ has the same radius (cut at $R - i$), so expanding starts from there. $R$ only moves right, so all expansions together cost $O(n)$.
**Modify / tips:** longest palindrome: $i$ = argmax `p[i]`, answer is `s.substr((i - p[i]) / 2, p[i])`. Number of palindromic substrings = sum of `(p[i] + 1) / 2`. `s[l..r]` (0-indexed) is a palindrome iff `p[l + r + 2] >= r - l + 1`. The characters `^`, `#`, `$` must not appear in `s`, else change the sentinels.
**Time:** $O(n)$

21 lines

```
vector<int> manacher(string s) { // p[] on t
    if (s.empty()) return {};
    string t = "^";
    for (char c : s) {
        t += '#';
        t += c;
    }
    t += "#$";
    int n = t.size();
    vector<int> p(n, 0);
    int C = 0, R = 0; // rightmost palindrome: centre, end
    for (int i = 1; i < n - 1; i++) {
        if (R > i) p[i] = min(R - i, p[2 * C - i]); // mirror
        while (t[i + 1 + p[i]] == t[i - 1 - p[i]]) p[i]++;
        if (i + p[i] > R) {
            C = i;
            R = i + p[i];
        }
    }
    return p;
}
```